Building data course — Week 1 — find your way around (read-only)

sparql-playground / lessons

Generated for print and offline reading

Week 1 — find your way around (read-only)

Five daily lessons. Each day has two scripts that compute the same thing: `dayNN_pandas.py` reads the CSV export into pandas, `dayNN_sql.py` runs SQLite queries on the same rows (`uv run python lessons/course.py setup` loads them once). Run both with `uv run python lessons/course.py run N` and compare the printouts. The scripts are in the repo next to each lesson. Keep notes under the ignored `student-work/` folder.

Contents

- Day 1 — Open the data; what one row is
- Day 2 — SELECT, WHERE, ORDER BY, LIMIT on a few columns
- Day 3 — Time: ISO timestamps, date functions, ranges, local time
- Day 4 — Summary stats: COUNT, MIN, MAX, AVG, SUM
- Day 5 — GROUP BY and HAVING: per day, per month, and the Week 1 challenge

---

Day 1 — Open the data; what one row is

Goal: open AHU_1's export in pandas and in SQLite, and say in one sentence what a row means.

Read

An HVAC controller measures things (temperatures, pressures, fan status) and a **historian** saves a reading every few minutes. Building 50's historian exported one CSV per air handler (AHU): `data/BUILDING_50/AHU_1/history_wide.csv` and the same for `AHU_2`. This week you work with AHU_1, and bring in AHU_2 only where it helps.

The CSV is a **wide** table:

- **One row is one moment**: a timestamp, every five minutes, in UTC.
- **One column is one point**: a sensor, a setpoint, a status or a command. The cell holds that point's value at that moment.

So `timestamp_utc = 2026-03-16 00:40:00+00:00` and `outside_air_temp_f = 69.0327` in the same row means: at 00:40 UTC the outside air sensor read 69.0327 °F. AHU_1's file has 84 columns (the timestamp plus 83 points) and 35,536 rows. (The opposite shape, one row per point per timestamp, is called **long**. Day 11 converts between the two, and Week 5's API stores long data.)

Two scripts open the same data two ways:

	day01_pandas.py
open	`pd.read_csv("data/BUILDING_50/AHU_1/history_wide.csv`
how many rows	`df.shape` (rows, columns)

	day01_pandas.py
what are the columns	`df.columns`
first rows	`df[show].head(5)`

What `setup_db.py` does. `uv run python lessons/course.py setup` runs `lessons/setup_db.py`, which reads each CSV and writes one SQLite table: `ahu_1`, `ahu_2` (and `fdd_events`, which Day 14 uses). Nothing is transformed. The columns keep their names and their order, `timestamp_utc` is stored as the same text as in the file and is the `PRIMARY KEY`, and every other column is `REAL`. So the pandas script and the SQL script look at identical rows. SQLite is a whole SQL database in one file: no server, no login, and Python ships with it (`import sqlite3`).

What the columns mean. Next to each CSV sits `columns.csv`: one line per column with the historian's point name (`bld_50_AHU_1 AHU1 OA-T`), its unit and a rough role. Names like `outside_air_temp_f` are cleaned-up versions of those. Units are US units: °F, inches of water (`inwc`), percent 0–100, and statuses are 0 or 1 stored as numbers. `columns.csv` is a file, not a table in the database; open it with `pd.read_csv` when you want the full names.

Why the column names are typed into code. All of Weeks 1–4 write names like `outside_air_temp_f` and `supply_fan_status` straight into the scripts. That is how a first analysis looks, and it is fine for one building. It also means the code only works on exports that use exactly those names: AHU_2's file has 67 columns, and calls its outside-air damper `mad_c_pct` where AHU_1 has `mad_c`. Week 5 replaces typed names with a model that says which column is the duct static pressure, so the same rule runs on any air handler.

Run

```
uv run python lessons/course.py setup        # once
uv run python lessons/course.py run 1
```

That runs `day01_pandas.py`, then `day01_sql.py`. Read them side by side: each numbered comment in one has the same number in the other. Both print:

```
Rows: 35536
Columns: 84
First column names: ['timestamp_utc', '50ahu1aalrm',
'bld_p_setpoint_inwc', 'bld_pressure_inwc',
'chw_valve_pct']
            timestamp_utc  outside_air_temp_f  da_p_inwc
da_p_setpoint_inwc
2026-03-16 00:40:00+00:00              69.0327   0.024594
1.2
2026-03-16 00:45:00+00:00              69.0327   0.024594
1.2
...
```

35,536 rows × 5 minutes is about 123 days. The first column is the timestamp; the other 83 are points.

Two things to notice:

- The outside air reading repeats: `69.0327` row after row. A historian often keeps the last value until the sensor changes enough, so repeated values are normal (Day 15 comes back to this).
- `read_csv` pulled all 84 columns into memory (about 24 MB). Fine here. From Day 2 on the scripts load only the columns they use (`usecols=[...]`).

Try it

1. How many columns does AHU_2 share with AHU_1? Which non-zone columns does only one of them have?
2. How many of AHU_1's columns are zone temperatures (names starting `zone_t50_`)?

Answers

1. 34 columns are common, 50 are only in AHU_1 and 33 only in AHU_2. Outside the zone temperatures, AHU_1 alone has `50ahu1aalrm`, `mad_c` and a long auto-generated `hu_50_duration_...` column; AHU_2 alone has `50ahu2aalrm` and `mad_c_pct`. The `mad_c`/`mad_c_pct` pair is the same kind of point (outside air damper) under two names: a rule written with a typed column name would work on one AHU and not the other.

   ```
   ahu_1 = pd.read_csv("data/BUILDING_50/AHU_1/history_wide.csv",
            nrows=0)  # nrows=0: header only
   ahu_2 = pd.read_csv("data/BUILDING_50/AHU_2/history_wide.csv",
            nrows=0)
   a = set(ahu_1.columns)
   b = set(ahu_2.columns)
   print(len(a & b), sorted(a - b), sorted(b - a))
   ```

   In SQL the same comparison is `SELECT name FROM pragma_table_info('ahu_1') EXCEPT SELECT name FROM pragma_table_info('ahu_2')`.

2. 47 of the 84 (31 in AHU_2). In pandas: `sum(c.startswith("zone_t50_") for c in df.columns)`; in SQL, count the rows of `pragma_table_info('ahu_1')` whose `name LIKE 'zone%'`. Most of an AHU's columns are readings from the zones it serves, not from the AHU itself.

Why it matters

Every analysis starts with "what is one row, and what is one column?". Here a row is a moment and a column is a point, which makes questions about one moment easy (compare the fan and the duct static in the same row) and questions about one point easy (take a column). What this table does not say is what each column *is*: `da_p_inwc` is the duct static pressure only because `columns.csv`, and some building knowledge, say so. Closing that gap is the point of Week 5.

Day 2 — SELECT, WHERE, ORDER BY, LIMIT on a few columns

Goal: pull a few columns of AHU_1, filtered, sorted and trimmed, in pandas and in SQL, and pass values to SQL safely.

Read

The parts of a query, in the order you write them, and the pandas that does each job:

```
SELECT  columns            -- what to show
df[["timestamp_utc", "outside_air_temp_f"]]
FROM    table              -- where from              (the
DataFrame you loaded)
WHERE   condition          -- which rows
df[df.da_p_inwc > 1.5]
ORDER BY column [DESC]     -- sorted how
.sort_values("outside_air_temp_f", ascending=False)
LIMIT   n                  -- how many
.head(n)
```

Today's columns, all from AHU_1's table `ahu_1`:

Column	What it is	Unit
`outside_air_temp_f`	outside air temperature	°F
`da_p_inwc`	duct static pressure (the pressure the supply fan builds in the supply duct)	inches of water
`da_p_setpoint_inwc`	the pressure the controller is trying to hold	inches of water
`supply_fan_status`	1 when the supply fan is running, 0 when off	0/1

Text values go in single quotes in SQL. Double quotes are for column and table names. To combine conditions use `AND` / `OR` in SQL and `&` / `|` in pandas. Each condition needs its own parentheses in pandas, because `&` binds tighter than `>`.

Parameters, not f-strings. In Python you'll be tempted to build SQL with an f-string: `f"... WHERE timestamp_utc = '{ts}'"`. Don't. If `ts` ever comes from a user, a file or an API request, a value like `x' OR '1'='1` rewrites your query (**SQL injection**), and even with trusted values you get quoting bugs. Instead leave a placeholder, `?` or a name like `:limit`, and pass the value separately. The database receives the SQL text and the value as two different things, so a value can never be mistaken for SQL. It also lets SQLite reuse the compiled query. Every query in this course passes values as parameters; only column and table names (which cannot be parameters) are written into the SQL text, and those are typed by you, never taken from input.

Where the work happens. `timestamp_utc` is the table's `PRIMARY KEY`, and SQLite keeps an index on it: a sorted B-tree. So `WHERE timestamp_utc >= :start AND timestamp_utc < :end` is a **seek**: walk the tree from the root to the first matching key, O(log n), then read the k matching rows in order. pandas has no such luxury on a plain column. `df["timestamp_utc"] >= start` compares every one of the n rows and builds a True/False array of length n: O(n). (A DataFrame whose index is sorted can do binary search too, but you pay for the sort, and the rows are already in memory.) A filter on `da_p_inwc` has no index in either tool, so both scan all n rows. `EXPLAIN QUERY PLAN` in front of a query shows which one SQLite chose (Try it 4).

A sort is different again: `ORDER BY ... LIMIT 1` only needs the biggest row, and `sort_values(...).head(1)` sorts all n rows to get it, O(n log n), where a single pass for the maximum would do. (`nlargest(1)` is the pandas way to ask for just that.) At 35,000 rows nobody notices. At a hundred million rows it is the whole job.

Run

```
uv run python lessons/course.py run 2
```

That runs `day02_pandas.py`, then `day02_sql.py`. Each step maps like this:

Step	day02_sql.py	day02_pandas.py
2. pick columns, first three	`SELECT timestamp_utc, outside_air_temp_f ... ORDER BY timestamp_utc LIMIT 3`	`df[["timestamp_utc", "outside_air_temp_f"]].head(3`
3. coldest	`ORDER BY outside_air_temp_f, timestamp_utc LIMIT 1`	`sort_values(["outside_air_tem "timestamp_utc"]).head(1)`
4. filter + count	`WHERE da_p_inwc > :limit` + `COUNT(*)`	`df[df["da_p_inwc"] > LIMIT_IN` then `len(...)`
5. two conditions	`WHERE supply_fan_status = 1 AND da_p_inwc < 0.5`	`df[fan_on & static_low]`

In pandas, `df["da_p_inwc"] > 1.5` makes a column of True/False, one per row, and `df[...]` keeps the True rows. SQL's `WHERE` drops the rows and `COUNT(*)` counts what is left. Same answer; SQL never builds the 35,536-element mask.

Adding `timestamp_utc` as a second sort key makes "coldest" deterministic: several readings can tie, because the sensor value only changes now and then, and without a tiebreaker SQL and pandas are free to return different ones.

Both scripts print:

```
            timestamp_utc  outside_air_temp_f
2026-03-16 00:40:00+00:00             69.0327
2026-03-16 00:45:00+00:00             69.0327
2026-03-16 00:50:00+00:00             69.0327
Coldest reading:
            timestamp_utc  outside_air_temp_f
2026-03-17 09:40:00+00:00             17.8308
Readings with static above 1.5 inH2O: 8336
Fan on but static below 0.5 inH2O: 782
```

782 readings with the fan running but almost no duct pressure: about 65 hours, well below any normal setpoint. That is a first hint of Day 14's FC1 rule.

The SQL script passes the 1.5 limit as the `:limit` parameter, never pasted into the SQL text. The second query writes its numbers (1 and 0.5) straight into the SQL: that is fine for numbers you typed yourself, and Try it 2 turns them into parameters.

Try it

1. What was the **hottest** outside air reading, and when? (Flip step 3's sort.)
2. Step 5's SQL writes `1` and `0.5` into the query text. Turn `0.5` into a parameter, the way step 4 passes `:limit`, and check you still get 782.
3. Print the five most recent timestamps, newest first.
4. Put `EXPLAIN QUERY PLAN` in front of two queries: one day of readings by `timestamp_utc` (`>= '2026-07-01 00:00:00+00:00' AND < '2026-07-02 00:00:00+00:00'`), and step 4's `da_p_inwc > 1.5`. What does SQLite say it does for each?
5. Count the rows `WHERE timestamp_utc = :ts` with `ts = "x' OR '1'='1"` passed as a parameter. Then paste the same value into the SQL with an f-string. What changes?

Answers

1. 98.86 °F at `2026-07-14 21:40:00+00:00`: `ORDER BY outside_air_temp_f DESC, timestamp_utc LIMIT 1` in SQL, `sort_values(["outside_air_temp_f", "timestamp_utc"], ascending=[False, True]).head(1)` in pandas.

```
sql = "SELECT COUNT(*) FROM ahu_1 WHERE
          supply_fan_status = 1 AND da_p_inwc < :low"
fan_on_low = conn.execute(sql, {"low":
          0.5}).fetchone()[0]
```

Still 782. In pandas the 0.5 is already an ordinary Python value.

3. Flip the sort: `ORDER BY timestamp_utc DESC LIMIT 5` in SQL, `df.sort_values("timestamp_utc", ascending=False).head(5)` in pandas (or just `df.tail(5)`). The newest row is `2026-07-17 09:55:00+00:00`. SQL's `DESC` on the primary key reads the B-tree backwards: no sort at all.
4. `SEARCH ahu_1 USING COVERING INDEX sqlite_autoindex_ahu_1_1 (timestamp_utc>? AND timestamp_utc<?)` for the day: the B-tree seek. The index has an odd name because `PRIMARY KEY` on a text column makes SQLite build an automatic index. For `da_p_inwc > 1.5` it says `SCAN ahu_1`: no index, so every row is read.
5. As a parameter: 0 rows. The value is treated as a plain (odd) timestamp that matches nothing. Pasted in with an f-string, the `WHERE` becomes "always true" and you get all 35,536 rows: SQL injection.

Why it matters

Almost every analysis step is "some columns, some rows, in some order, not too many". Getting comfortable with that shape in both tools means you can read other people's queries and DataFrame code as the same thing. Passing values as parameters is a habit worth forming now: in Week 5 the column and series you ask for will come from a model lookup, not from your own keyboard, and a value that came from somewhere else must never be able to become part of the query.

Day 3 — Time: ISO timestamps, date functions, ranges, local time

Goal: turn `timestamp_utc` into dates and date parts, select a day, and know where each tool's time handling bites.

Read

`timestamp_utc` is **ISO 8601 text**: `2026-03-16 00:40:00+00:00` is year-month-day, a space, hour:minute:second, and a UTC offset (`+00:00` means UTC). Historians store UTC because it has no daylight saving jumps and sorts the same way as the clock.

Because every value has the same fixed-width format, **sorting the text is sorting by time**. That is why Day 2's range query worked on raw text. But text is still text: to get *parts* of the date (the hour, the weekday, the month) or to do arithmetic, you need real date handling:

- **pandas** parses once: `df["time"] = pd.to_datetime(df["timestamp_utc"], utc=True)` gives a datetime column (`datetime64[ns, UTC]`) with the whole `.dt` toolbox.
- **SQLite** has no date type at all. `datetime()`, `date()`, `strftime()` and `unixepoch()` read the text and compute on it **row by row**, every time you call them.

How the two line up for `2026-03-16 00:40:00+00:00`:

SQL (on the text column)	pandas (`t = pd.to_datetime(..., utc=True)`)	Result
`datetime(timestamp_utc)`	`t.dt.strftime("%Y-%m-%d %H:%M:%S")`	`2026-03-` `00:40:00` (offset dropped)
`date(timestamp_utc)`	`t.dt.strftime("%Y-%m-%d")`	`2026-03-`

SQL (on the text column)	pandas (`t = pd.to_datetime(..., utc=True)`)	Result
`strftime('%H', timestamp_utc)`	`t.dt.hour`	`00` (text in SQL, int in pandas)
`strftime('%w', timestamp_utc)`	`(t.dt.dayofweek + 1) % 7`	`1` (a Monday)
`unixepoch(timestamp_utc)`	`(t - epoch).dt.total_seconds()`	`17736216`

`unixepoch` is **Unix epoch seconds**: seconds since 1970-01-01 00:00:00 UTC. Subtracting two of them gives seconds between readings, which is how the scripts check there are no gaps: 5-minute steps from the first to the last reading need `span_s / 300 + 1 = 35,536` rows, and there are exactly that many.

Weekday gotcha. SQLite's `%w` counts **Sunday = 0** and returns **text**; pandas' `dayofweek` counts **Monday = 0** and returns an integer. 2026-03-16 was a Monday: 1 in SQL, 0 in pandas. The pandas script converts with `(dayofweek + 1) % 7` (step 7), and the SQL script wraps `%w` in `CAST(... AS INTEGER)`.

Date ranges: half-open. To select a day, write `start <= time < end` with `end` = the next midnight. Consecutive days then never share a reading, and "the day" needs no thought about 23:59:59.999. Closed ranges (`BETWEEN`, `<=`) are where off-by-one days come from. They also bite on short text: `BETWEEN '2026-07-01' AND '2026-07-01'` returns **zero** rows, because `'2026-07-01 00:00:00+00:00'` sorts *after* `'2026-07-01'` (a longer string with the same start is greater). The scripts build bounds as full timestamps, `2026-07-01 00:00:00+00:00`, which in pandas is simply `str(pd.Timestamp("2026-07-01", tz="UTC"))`.

Don't filter with a function of the column. `WHERE date(timestamp_utc) = '2026-07-01'` returns the same 288 rows as the range, but SQLite has to compute a date for every row

instead of seeking. The range form lets it use the index: a B-tree seek, O(log n + k). The function form is a scan, O(n). The plans (`EXPLAIN QUERY PLAN` in front of each query):

```
SEARCH ahu_1 USING COVERING INDEX sqlite_autoindex_ahu_1_1
(timestamp_utc>? AND timestamp_utc<?)
SCAN ahu_1 USING COVERING INDEX sqlite_autoindex_ahu_1_1
```

(Both read the index, since it holds the whole key; `SEARCH` jumps to the range, `SCAN` walks all of it.) The pandas twin of the mistake is formatting the whole column as strings just to compare dates, which is a Python-level pass over every row; comparing the datetime column to two bounds is one vectorized comparison. With 35,000 rows both are instant; with a month of 1-second data from a thousand points the difference is minutes.

Time zones. Store UTC, convert at the edges. pandas converts to a named zone, daylight saving included: `t.dt.tz_convert("America/Chicago")`. SQLite has only `'localtime'`, the zone of **whatever machine runs the query**, so the same query gives different answers on different computers. That is a reason to keep zone conversion in pandas or application code. Building 50 is in the Central zone. Its data (March to July) is all daylight saving time, UTC−5; the spring change was 8 March, before the data starts, and the autumn one is in November, after it ends.

Run

```
uv run python lessons/course.py run 3
```

That runs `day03_pandas.py`, then `day03_sql.py`. pandas turns the text into real timestamps first (`pd.to_datetime(..., utc=True)`); SQL compares the ISO text directly, which works because every value has the same format. Both print:

```
First reading: 2026-03-16 00:40:00+00:00
Last reading:  2026-07-17 09:55:00+00:00
Readings on 2026-07-01: 288 (24 hours x 12 per hour)
Readings on 2026-03-16: 280
Rows if every 5-minute step is there: 35536
Rows actually there:                  35536
Weekday of the first reading (0 = Sunday): 1
```

288 = 24 hours × 12 readings an hour: a full day of 5-minute data. The first day, 2026-03-16, has 280, because the data starts at 00:40, not midnight, so eight readings never logged. Steps 4 and 5 are half-open ranges: `>=` the start, `<` the next midnight. No rows are missing between the first and last reading.

Only pandas can convert to a named time zone. Try it 1 does that.

Try it

1. When was the hottest reading in `America/Chicago` time, and is that a plausible hour for a hot day?
2. How many readings does June 2026 have, as a half-open range? Why would `<= '2026-06-30'` be wrong?
3. When did the fan first start on 2026-06-04, in UTC and in local time? (Read `supply_fan_status` too, keep that day's rows with step 4's range, and take the first row where the status is 1.)

Answers

1. 16:40 (4:40 pm) local on 14 July, `21:40 UTC` minus five hours. Mid-afternoon is when the outdoor temperature peaks, so yes. In pandas, read `outside_air_temp_f` too, then:

   ```
   hottest = df.sort_values("outside_air_temp_f").tail(1)
   print(hottest["time"].dt.tz_convert("America/Chicago"))
   ```

   SQLite has no named time zones (only `'localtime'`, the zone of whatever machine runs the query).

2. 8,640 (30 days × 288), with `timestamp_utc >= '2026-06-01 00:00:00+00:00' AND timestamp_utc < '2026-07-01 00:00:00+00:00'`. `<= '2026-06-30'` compares the *text*, and every reading on the 30th (`2026-06-30 00:05:00+00:00` and so on) sorts after `'2026-06-30'`, so the whole last day would be missing.

3. 09:05 UTC, which is 04:05 local (UTC−5). On 1 April it was 09:00 UTC, 04:00 local. The fan starts around 4 am local: an early start (a warm-up or cool-down before occupancy?), the kind of thing you only see once you read the time in local terms.

Why it matters

Building people think in local time (“the fan should start at 6 am”). Databases should store UTC. Converting at the edges and noticing what a UTC timestamp is in local time is a daily skill: a schedule fault looks like a five-hour mistake if you forget. The half-open range and the “never wrap the indexed column in a function” rules are the two habits from today that you will use in every later week.

Day 4 — Summary stats: COUNT, MIN, MAX, AVG, SUM

Goal: summarize a column in one row, in pandas and in SQL, and turn a status point into run hours.

Read

Aggregate functions fold many rows into one value. pandas has a method for each:

SQL	pandas (on a column `v`)	Notes
`count(*)`	`len(df)`	rows
`count(col)`	`v.count()`	rows where the value is not NULL / NaN
`min(col)`, `max(col)`	`v.min()`, `v.max()`	ignore NULLs / NaN
`avg(col)`	`v.mean()`	mean of the non-NULL values
`sum(col)`	`v.sum()`	total (SQL: NULL for no rows; pandas: 0)
`round(x, n)`	`round(x, n)`	for reading, not storage. **Never round before comparing**

Without `GROUP BY` (tomorrow), an aggregate query returns exactly one row.

NULLs. A missing reading is `NULL` in SQL and `NaN` in pandas. Both skip it in `min`, `max`, `avg` and `sum`, and both agree that `count(col)` counts only the values that are there, while `count(*)` counts rows. So the mean of `1, <missing>, 3` is 2.0, not 4/3. The Building 50 columns have no gaps, so try it on three made-up values: `pd.Series([1, None, 3]).mean()` in pandas, `SELECT AVG(x) FROM (SELECT 1 AS x UNION ALL SELECT NULL UNION`

`ALL SELECT 3)` in SQL. Both give 2.0. In real data a flat-lined or missing sensor often shows up as NULLs, or as a suspicious constant like `0` or `-999`; only the first is skipped for you.

Run hours. A status point is 1 when running and 0 when off. With one reading every 5 minutes, `sum(status) × 5 / 60` is the **run hours**. Use `60.0`, not `60`, in SQL: with two integers SQLite divides as integers and drops the remainder (`7 * 5 / 60` is `0`). Python 3's `/` is always true division; its `//` is the one that floors. Different engines, different gotchas.

How each engine does it. Both are one pass over the rows, O(n). SQLite computes every aggregate in the `SELECT` list in the *same* pass, while it streams the rows off the table, and returns a single row. pandas works on columns in memory: each of `min`, `max`, `mean` is its own fast, vectorized pass (a compiled C loop, not a Python `for`), and the rows had to be loaded into Python first. So SQL moves one row of results; pandas moves all the input. Nothing here is faster in an *algorithmic* sense. The aggregate is O(n) either way, and no index helps with `avg()`. What differs is how much data crosses from the database into your program.

Run

```
uv run python lessons/course.py run 4
```

That runs `day04_pandas.py`, then `day04_sql.py`. Both print:

```
Outside air temperature (F): average 61.28 min 17.8308 max
98.8608
Supply fan run hours: 1574.4
Fraction of time the fan ran: 0.532
```

How each step maps across:

Step	day04_sql.py	day04_pandas.py
2. three stats in one pass	`SELECT AVG(c), MIN(c), MAX(c) FROM ahu_1`	`df[c].mean()`, `df[c].min()`, `df[c].max()`

Step	day04_sql.py	day04_pandas.py
3. run hours	`SUM(supply_fan_status) * 5 / 60.0`	`df["supply_fan_status"].sum() * 5 / 60`
4. fraction of time on	`AVG(supply_fan_status)`	`df["supply_fan_status"].mean`

The dataset covers about 2,961 hours (35,536 readings × 5 minutes), so AHU_1's fan ran 1,574 of them, 53.2% of the time. (The average of a 0/1 column is the fraction of rows that are 1.)

The averages are printed rounded, but the checks behind these lessons compare the raw values to 1e-9: rounding before you compare turns "equal" into "equal to 3 decimals".

Try it

1. What are AHU_2's average outside air temperature and its duct static/setpoint averages? How do they compare with AHU_1's? (Add AHU_2 to the SQL with a second `FROM`, or `UNION ALL`.)
2. How many hours was AHU_1 scheduled occupied? Occupancy is column `occ_c` (1 = occupied). Compare with the fan's run hours.

Answers

1. AHU_2: outside air 63.42 °F (AHU_1: 61.28), duct static 0.785 inH2O against a setpoint average of 0.775. The two outside-air averages differ by 2 °F even though both AHUs sit in the same building: they are different sensors in different places, so even the "same" quantity needs checking before you compare it. And AHU_2's average static is at its setpoint, where AHU_1's static is far below its (1.366). That gap is today's hint at a problem on AHU_1. But it includes all the hours the fan was off, when static is near zero. Day 8 asks the fair question: what is static *while the fan runs*?

```
SELECT avg(outside_air_temp_f), avg(da_p_inwc),
       avg(da_p_setpoint_inwc) FROM ahu_2
```

2. 1,518.0 occupied hours (`sum(occ_c) * 5 / 60.0`) against 1,574.4 fan hours: the fan runs 56 hours more than the schedule says the building is occupied, on top of any hours it runs for warm-up or night purge. (AHU_2: 1,519.25 occupied hours, 1,499.6 fan hours.) Whether that is waste or sensible depends on the sequence of operations, and it is the kind of question Week 3's gating makes precise.

Why it matters

Averages over everything hide what matters. A static pressure reading near 0 because the fan is off pulls the average down just like a real problem does. Most of fault detection is asking "the average **of what, when**?" That is filtering (Days 8–9) and gating (Week 3). Run hours are the first example of a number a building owner actually reads: hours the fan ran, hours a fault was active, hours of energy at a given load.

Day 5 — GROUP BY and HAVING: per day, per month, and the Week 1 challenge

Goal: one summary row per group, keep only the groups you care about, in pandas and in SQL, and answer a three-measure question per month.

Read

`GROUP BY` splits rows into groups. Each aggregate is then computed once per group:

```
SELECT group_columns, aggregates
FROM ...
WHERE   row filter           -- before grouping
GROUP BY group_columns
HAVING  group filter          -- after grouping, can use
aggregates
ORDER BY ...
```

`WHERE` throws away rows before they are grouped. `HAVING` throws away whole groups after. You can't put `avg(...)` in a `WHERE`, because no group exists yet at that point.

You can group by an expression, such as the date: `GROUP BY date(timestamp_utc)` gives one row per day, and `strftime('%Y-%m', timestamp_utc)` one row per month (Day 3's date functions). SQLite lets you use a column alias (`AS day`) in `GROUP BY` and `ORDER BY`. The pandas version of each clause:

SQL	pandas
`WHERE cond`	`df[mask]` before the `groupby`
`GROUP BY key`	`df.groupby(key)` (key can be a column or a computed Series, like a formatted date)
`avg(c)`, `count(*)`, `sum(c)`	`.agg(avg_f=("c", "mean"), n=("c", "size"), ...)`

SQL	pandas
`HAVING count(*) < 288`	filter **after** the groupby: `grouped[grouped.n < 288]`
`ORDER BY x DESC LIMIT 1`	`.sort_values("x", ascending=False).head(1)`
`sum(da_p_inwc > 1.5)`	`(df.da_p_inwc > 1.5).sum()`

So `HAVING` is not a special pandas feature. It is just an ordinary filter on the small table `groupby` gives you. `WHERE` and `HAVING` differ in SQL only because of clause order; in pandas the order of your statements says it.

Counting with a comparison. In SQLite a comparison such as `da_p_inwc > 1.5` is `1` when true and `0` when false, so `sum(da_p_inwc > 1.5)` counts the rows where it holds, in the same pass as any other aggregate. pandas does the same with a boolean column (`True` is `1` when you `sum`). That makes "count the readings that meet a condition, per group" a one-liner, and it is the shape most fault-detection summaries take.

How each engine groups. pandas **hashes**: one pass over the rows, each row dropped into a bucket for its key, O(n). SQLite either **sorts** the rows by the key (into a temporary B-tree) and sums runs of equal keys, O(n log n), or, if the rows already come out in key order, streams them without a sort. `timestamp_utc` is the primary key, so rows come out in time order, but a *day* is computed from it and SQLite doesn't know the computed values are in order, so `GROUP BY date(timestamp_utc)` needs the temporary B-tree. `EXPLAIN QUERY PLAN` in front of the query says so: `SCAN ahu_1 | USE TEMP B-TREE FOR GROUP BY`. At 35,536 rows this costs nothing you can notice.

Run

```
uv run python lessons/course.py run 5
```

That runs `day05_pandas.py`, then `day05_sql.py`. Each step maps like this:

Step	day05_sql.py	day05_pandas.py
2. month and day labels	`strftime('%Y-%m', timestamp_utc)`, `date(timestamp_utc)`	`time.dt.strftime("%Y-%m")`, `"%Y-%m-%d"`
3. one row per month	`GROUP BY month` + `AVG`, `SUM`, `SUM(da_p_inwc > 1.5)`	`df.groupby("month").agg(...)`
4. hottest day	`GROUP BY day ORDER BY avg_f DESC LIMIT 1`	`groupby("day")[...].mean()`, then `idxmax()`
5. short days	`GROUP BY day HAVING COUNT(*) < 288`	`groupby("day").size()`, then a filter on `n < 288`

Both print:

```
  month  avg_oat_f  fan_run_hours  static_above_1_5
2026-03       41.5          182.7                 0
2026-04       53.0          341.9                 0
2026-05       59.8          331.7                 0
2026-06       71.3          411.6              4721
2026-07       80.0          306.6              3615
Hottest day: 2026-07-01 average (F): 88.3
       day   n
2026-03-16 280
2026-07-17 120
```

1 July (88.30) edges out 2 July (88.25) by 0.04 °F, so the two look tied at one decimal. The sort used the unrounded averages. Round for display only.

The two short days are the first (2026-03-16, starts at 00:40, 280 readings) and the last (2026-07-17, ends at 09:55, 120 readings). There are no gaps in between. Real historians usually aren't this tidy, so checking is always worth it. The short days need `HAVING` in SQL because the condition is on a count, which only exists after grouping; in pandas it is an ordinary filter on the small grouped table.

Try it

1. AHU_1's fan run hours per month: which month had the most?
2. Which months had any readings with duct static above 1.5 inH2O? (Use the Week 1 challenge's output.)
3. Show only the months where AHU_1's fan ran more than 400 hours. Where does that condition go, `WHERE` or `HAVING`?

Answers

1. June, with 411.6 hours (March 182.7, April 341.9, May 331.7, July 306.6; March and July are partial months).

2. Only June (4,721 readings on AHU_1) and July (3,615). In March, April and May AHU_1's static never went above 1.5. In pandas: filter the monthly table with `monthly[monthly["static_above_1_5"] > 0]`.

3. `HAVING`, because it is a condition on an aggregate (`sum(...)`) of the group:

```sql
SELECT strftime('%Y-%m', timestamp_utc) AS month,
         sum(supply_fan_status) * 5 / 60.0 AS
         run_hours
FROM ahu_1
GROUP BY month
HAVING sum(supply_fan_status) * 5 / 60.0 > 400
```

   Only `2026-06` comes back. In pandas it is `monthly[monthly["fan_run_hours"] > 400]`, after the groupby.

Week 1 challenge

For each month, report **the average outside air temperature, the fan run hours, and the number of readings where duct static was above 1.5 inH2O**.

This is step 3 in both scripts, the first table they print. Try writing it yourself first, in each tool, before you read the scripts' versions. What to notice:

- In the old long layout the three measures lived in three different series, and one query could not easily combine them. In the wide table all three are columns of the same row, so one `GROUP BY` and three aggregates answer the question: `AVG(outside_air_temp_f)`, `SUM(supply_fan_status) * 5 / 60.0`, `SUM(da_p_inwc > 1.5)`.
- Then do it for **both AHUs** in one table: ten rows, two AHUs × five months. `equipment_id` is not a column of either table, so add it as you stack them: `SELECT 'AHU_1' AS equipment_id, ... FROM ahu_1 UNION ALL SELECT 'AHU_2', ... FROM ahu_2` in SQL, `df["equipment_id"] = "AHU_1"` on each DataFrame and `pd.concat` in pandas. Then group by two keys, `equipment_id` and `month`.

Static above 1.5 shows up only once it is warm, in June and July, for both AHUs. Next week's questions are about *why*, and whether that is a problem: AHU_1's average setpoint is 1.37 inH2O, so a reading above 1.5 might be the system doing exactly what it was asked.

Why it matters

A summary table per day or per month is the first thing a building owner will ask for, and `GROUP BY` plus a filter on the groups is how every such table is made. The group key does a lot of quiet work: group by a date you computed in UTC and "a day" starts at 7 pm local time in summer, which is a mistake you will not see in the numbers. Before judging a reading you also have to say *which* readings count (when the fan is on, when the building is occupied).

Reading — Week 1

The lessons first, then these for exact behavior.

- Python sqlite3 module
- pandas: comparison with SQL
- DB Browser for SQLite
- SQLite SELECT
- SQLite date and time functions
- SQLite aggregate functions
- RFC 9562: UUIDs (version 5 = name-based)